

JPMorgan Chase backend and integration work

Adam Bates | Software Engineer I / II, Asset & Wealth Management | February 2022 - February 2025

I built Java/Spring Boot services for order management and routing, independently implemented Kafka messaging within the trade-routing platform, and built its order-audit system. My work connected service behavior, diagnostic evidence, and production validation so the team could understand how an order moved through the system and investigate problems more effectively.

Responsibility and collaboration

I implemented the Kafka messaging that replaced the internal routing mechanism. Topic, key, and partition design involved a more senior developer, and some logic was pair-programmed. My services handled order management and routing within a broader financial platform; other teams and engineers owned the surrounding capabilities.

I also built the order-audit system end to end with Spring Boot, SQL, Liquibase, and documented APIs searchable by field. The append-only history supported engineering investigations as well as compliance and audit requests.

Decisions and validation

The routing change introduced asynchronous processing while retaining the existing order-state logic. I implemented idempotent consumers, per-order ordering, locking, and replay handling to prevent a retried message from processing an order twice. Those mechanisms addressed the application's order-processing behavior; they are not a claim that every external action can happen exactly once.

I built end-to-end order-lifecycle tests using Groovy, JUnit, and Mockito. I also partnered with another team's lead on performance testing during the Kafka upgrade. Production releases took place after Friday market close, with failover checks and monitoring through Splunk and Datadog.

Separately, I led an order-management database migration to AWS. I kept the source and destination synchronized, converted the schema and database-specific SQL, and validated the migrated data. The team reviewed the cutover plan before the move after market close.

Result and scope

After the Kafka upgrade, the platform had fewer failures and was easier to investigate. The audit system made bugs faster to identify by preserving the sequence of order events. The database migration completed without downtime during trading hours and improved cost, reliability, and security.

These are descriptions of my contribution within the team's platform. They do not imply ownership of the entire trading system, and the performance-testing work has no separately recorded result to claim.

github.com/B8Z | linkedin.com/in/b8z

Capital One: learning the system and making its behavior visible

Adam Bates | Senior Software Engineer, Bank Tech | February 2025 - August 2026

I joined Capital One without prior knowledge of its ATM architecture. I learned how the application, vendor platform, host, and hardware worked together, then used that understanding to lead the interface modernization team's technical work and independently design and build a Python validation platform.

The useful part of that knowledge was being able to explain what the system was doing and give other people a way to inspect it themselves.

Understanding the system

A customer journey crossed several systems and team boundaries. Understanding the interface meant understanding the transaction flow behind it, the host's behavior, and what happened at the device. I diagrammed transaction flows and architecture, investigated ATM journals, and worked with vendors and internal teams to trace problems across those boundaries.

In separate production incident work, I traced a reboot loop across application, host, device, and vendor layers and drove root-cause fixes with the teams involved. I also helped the modernization team learn the architecture as I directed its technical work.

Turning that understanding into a tool

One integration decision concerned the shared Vue.js library and the ATM transaction flows. I set the approach so the UI team could keep developing the library while the ATM team styled screens in CSS and kept the flows stable. That division let each team contribute within the system's constraints.

Product, design, legal, hardware, and engineering teams had different responsibilities and constraints. The validation project was assigned to me; I determined its architecture independently while working with those teams. The platform gave them a shared view of how the implemented customer journeys behaved.

I used Python and Playwright to run functional and visual checks on QA hardware. A run followed reachable customer flows, collected evidence, compared screens with their intended designs, and produced flow maps and remediation reports. The dashboard brought those outputs together for engineers and partner teams.

Visual and functional differences became structured defect reports supporting reviewed, vendor-compatible fixes, rebuilding, and revalidation. Engineering could act on the defect evidence, while product, legal, and hardware teams could inspect the implemented behavior in the same dashboard. The tool connected validation to the work people needed to do afterward.

Contribution and outcome

I independently designed and built the validation tooling. I also led the interface modernization team's technical work and implemented the customer interface, integrating a shared Vue.js library with the vendor platform. Other engineers built that library, and the wider ATM team owned releases and operations.

The platform automated validation and reporting for the implemented, reachable scenarios and helped other teams understand the software. The overall modernization remained in progress when I left. Visual checks did not establish complete accessibility compliance, and a successful validation run did not mean a production release had occurred.

github.com/B8Z | linkedin.com/in/b8z